Timing-Sensitive Non-Interference for Interrupt Handling, Compositionally and Mechanically

Willard Rafnsson
IT University of Copenhagen, Denmark
wilr@itu.dk

Dawit Tirore
IT University of Copenhagen, Denmark
dati@itu.dk

***Abstract*—Interrupt handling is a key part of an operating system, and a source of timing leaks. Servicing an interrupt takes time from whichever process was scheduled, so an attacker who watches a public process sees when a handler ran and for how long. If the interrupt is secret, so is that information. Such leaks arise in several operating systems [TODO: examples, with citations]. We use the process calculus of Rafnsson et al., whose timing-sensitive non-interference theorems compose, to model an operating system with interrupt handling as a single process, and prove in Rocq that ordinary interrupt handling leaks: a secret disk interrupt is observable through the schedule alone. A non-interfering policy must make *when* secret handlers run, and for how long, depend only on public events, while *which* secret handler runs may stay hidden. We model a policy that does so: secret handlers run only in time slices of fixed length, opened by the public timer handler, and runs no interrupt claims are filled by a dummy handler. We prove it timing-sensitive non-interfering for any scheduler and user processes that are themselves non-interfering. The implementation of this policy is carried out in separate work [TODO: cite X].**

***Index Terms*—non-interference, timing channels, interrupts, process calculi, compositional verification, Rocq**

I. Introduction

An operating system owns the devices, decides which process runs and when, and stands between processes that must not learn about each other. Whatever it lets one process observe about another is a potential leak, and that includes *when* things happen, not only what values are computed.

Interrupts are how an operating system learns that something happened: a disk transfer completed, a packet arrived, a timer expired. Some of these events are secret. Whether a particular process just received data from the disk may reveal what it is doing, so an operating system that keeps such an event secret has to hide that it happened at all.

Ordinary interrupt handling does not hide it. As soon as the interrupt can be serviced, its handler runs, displacing whatever process was scheduled; the handler takes a few steps, and then the displaced process resumes. Suppose a public process keeps issuing requests whose results an attacker can see. While the handler runs, those requests stop, and the attacker sees a gap in the stream exactly as long as the handler (Fig. 1). The interrupt has leaked through the *schedule*. The attacker needs no view of the schedule itself: the gap is in the public process's own output, as the user-view rows of Fig. 1 show. Timing leaks through interrupt handling of this kind arise in several operating systems [TODO: examples, with citations].

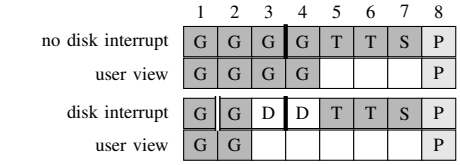


Fig. 1. Who runs at each step under ordinary interrupt handling, with and without a disk interrupt. G: public process, P: private process, S: scheduler, T, D: timer and disk handler. Shading is what the attacker sees: the step and its value (dark), only that it happened (light), or nothing (white). A solid bar marks the timer interrupt, a hollow bar the disk interrupt. The disk handler takes steps 3 and 4 from the public process, a gap the attacker sees. Under each run, its user view: only the two user processes, every other step blank. Here the gap can also be seen.

This is the example we follow throughout the paper. The secret is *whether a disk interrupt occurred*. A public process issues requests whose results the attacker sees, and a private process, the legitimate consumer of the disk event, reacts to it. The attacker may learn nothing about the disk interrupt. Sections III to V model this system, exhibit the leak, and remove it.

Leaks of this kind are the subject of information flow control [1], where the absence of a leak is stated as non-interference [2]: varying what the attacker may not see must not change what it does see. For timing leaks, "what it sees" must include when each output appears [3].

We use the process calculus of Rafnsson, Jia and Bauer [4], whose non-interference is timing-sensitive and whose theorems compose: if the parts of a process are non-interfering at suitable equivalences, so is the whole. We model an operating system with interrupt handling as one closed process in this calculus (Fig. 3): the user processes, the scheduler and the handlers run in a pool, and a global state decides at each step which of them runs. We prove in Rocq that ordinary interrupt handling is interfering (`model_immediate_not_NI`): the leak of Fig. 1 is a counterexample, in the user view as well (`model_immediate_userview_not_NI`).

What the attacker learns is *when* a secret handler runs and *for how long*. A non-interfering policy must therefore fix both by public events; only *which* secret handler runs may depend

on the secret. We model a policy that does so. Secret handlers run only inside time slices, each opened when the handler of the public timer completes and each holding a fixed number of handler runs; a run that no real interrupt claims is filled by a dummy handler, the NOP handler. A secret handler run then replaces a NOP run instead of displacing a user process. That this is safe is intuitive; proving it is not. The secret still changes the state: it sets a pending bit, decides which handler fills a run, and changes what the private user process does next. The proof must show that none of this ever decides whether, or for how long, a secret handler runs (Section V).

We prove this policy non-interfering (`model_sliced_NI`), even to an attacker who sees at every step which process ran, except which of the two secret handlers. The user view is a projection of this output, so the result carries over to it (`model_sliced_userview_NI`). Both policies are instances of a single generic model and differ only in their *interrupt policy*: the part that decides when handlers may run. The result is general in two ways. It holds for every scheduler and every pair of user processes that are themselves non-interfering. And it holds for every choice of the two numbers that fix the timing of the policy: the number of steps one handler run takes, and the number of handler runs that fit in one time slice.

The model fixes *what* the modified interrupt processing must do; how to implement it in an operating system is carried out in separate work [TODO: cite X].

a) Contributions:

- A mechanised, timing-sensitive non-interference proof for interrupt handling (Sections IV and V), and a mechanised counterexample for ordinary interrupt handling (Section IV-A). The proof is compositional: it applies one generic theorem per constructor of the model, which leaves two side conditions specific to interrupt handling, one of them hard (Section V-A).
- Requirements, read off the proof's obligations, on any implementation of interrupt handling that is to be non-interfering (Section V-E). The arrival of a secret interrupt may only record that it is pending. When secret handlers run, and for how long, must be fixed by public events, so the time reserved for them must be fixed in advance and always used up. And the interrupt controller must be split by secrecy: which interrupts are pending is secret, while the masks that decide what runs next are public. The implementation in [TODO: cite X] follows these requirements.
- A mechanisation of the calculus and its composition theorems. Our definitions depart from the original in one essential place: non-interference is stated over finite traces, which makes every proof an induction. We prove the definition equivalent to the original coinductive one, so our main results also hold in the original's terms (Section VI). We also add sum types, which the original lacks, and present its optional values and parts of the model as sums, proved equal to the mechanised definitions.

Section II recalls the calculus and the definition of non-interference. Section III presents the generic model and Section IV the two interrupt policies. Section V gives the proof, and Section VI describes the mechanisation. Sections VII and VIII discuss related work and limitations.

II. Background: Composable Timing-Sensitive Non-Interference

This section recalls the calculus and definitions of Rafnsson et al. [4], in the form we mechanise. The one essential difference is that non-interference is stated over finite traces; Section VI-A proves it equivalent to the original. We also add sum types, which the original lacks, and write optional values and the model's interrupts as sums; their equivalences are proved equal to the mechanised ones (Section VI-B).

A. Processes, traces and time

A process $p : \mathit{Proc}\ I\ O$ has an input type I and an output type O. It moves in one of two ways, by receiving an input, $p \overset{i}{\rightsquigarrow} p'$, or by emitting an output, $p \xrightarrow{o} p'$, in any order. Processes are built from seven constructors, whose reduction rules are given in Fig. 2. The operating system of Section III is a single closed term over them.

`out o` ignores its input and always emits `o`. `map f g p` translates inputs by `f` on the way into `p` and outputs by `g` on the way out. `sta f g v p` keeps a state cell `v`: an input `i` updates it to `f i v`, and `p` receives the new state together with `i`; an output `o` of `p` updates it to `g o v`, and the new state is emitted together with `o`. `swi b p` switches `p` on and off. While the phase `b` is `false` it emits `None` and `p` does not move; while it is `true` it emits `Some o` for an output `(c,o)` of `p`. The phase is toggled by the tag `c` and by the bit carried on each input. `par p1 p2` gives every input to both parts and emits both outputs as a pair. `loop p` feeds each output of `p` back to it as an input. `maybe p` makes the input optional: `None` leaves `p` unchanged. We write $A + 1$ for the optional values of A, with `Some v` and `None` its two injections.

Reduction is deterministic, and a process never blocks.

Time is counted in output steps: every output step is one tick of a clock the observer shares, and on every tick a process emits a value (in our models, a component that did not run emits `None`). Input steps take no time. This makes the setting timing-sensitive: a process that emits `o` one step late emits something else, say `None`, at the original step, so a delay is observed as a change of value.

B. Characterised equivalences and non-interference

Security is stated relative to a lattice of levels ℓ. Everything in this paper can be read with the two-point lattice $\mathsf{L} \sqsubseteq \mathsf{H}$: the attacker L, and H, who sees everything; the mechanisation is over any partial order with a least element, in particular any bounded lattice (Section VI). Each input and output type carries a *characterised equivalence* (`cEquiv`) $\doteq$, which says, for each level ℓ:

- $x \doteq_\ell y$: an observer at ℓ cannot tell x from y;

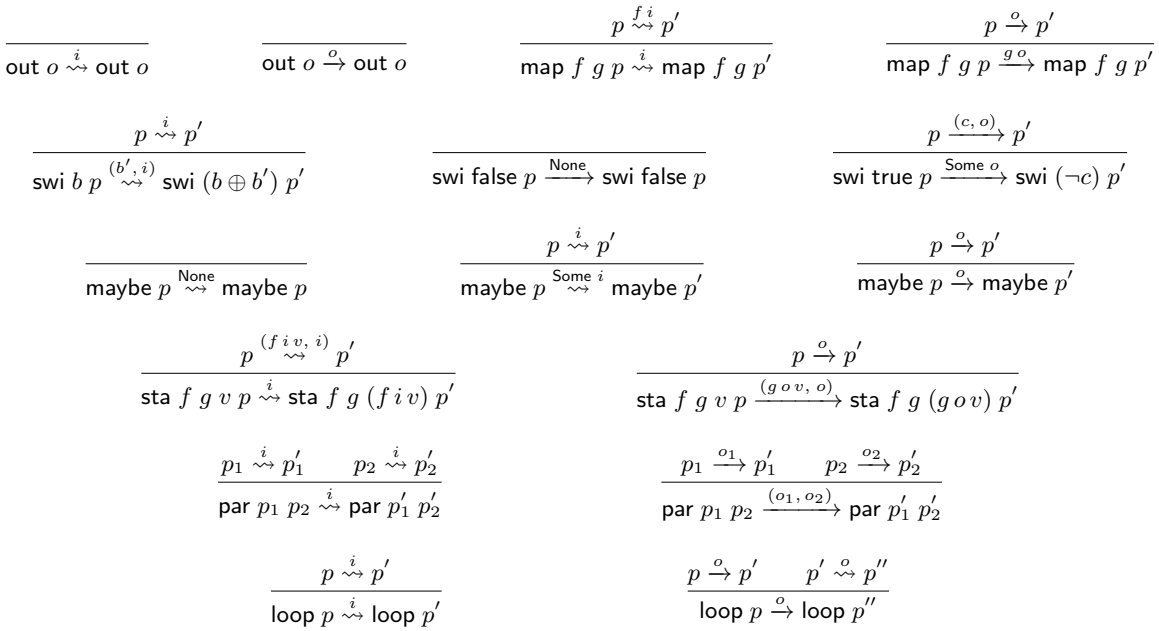


Fig. 2. Reduction rules (`reduceI`, `reduce0`). Wavy arrows are input steps, straight arrows output steps; $\oplus$ is exclusive or.

- $x \doteq_\ell \bullet$: an observer at ℓ does not see x at all.

This definition is an alternative to Rafnsson et al.'s, which adjoin $\bullet$ as a value; the two give the same non-interference (`NI_lEquiv`). $\doteq_\ell$ is an equivalence, and the unobservable values form one of its classes: they are related to each other and to nothing else. So at each level the type is partitioned into classes the observer cannot tell apart, and one of them, the *distinguished class*, holds the values the observer does not see. It is empty when the level hides nothing. We write $\overset{I}{\doteq}$ and $\overset{O}{\doteq}$ for the equivalences on inputs and outputs.

A trace t is a finite word of input effects $?i$ and output effects $!o$. We write ϵ for the empty trace and "." for concatenation. A process p *performs* t at ℓ, written $p \overset{t}{\dashrightarrow}_\ell$, if it has a run with the effects of t, outputs compared up to $\overset{O}{\doteq}_\ell$ (`Trace`):

$$\frac{}{p \overset{\epsilon}{\dashrightarrow}_\ell} \qquad \frac{p \overset{i}{\rightsquigarrow} p' \qquad p' \overset{t}{\dashrightarrow}_\ell}{p \overset{?i.t}{\dashrightarrow}_\ell}$$

$$\frac{p \xrightarrow{o'} p' \qquad o' \overset{O}{\doteq}_\ell o \qquad p' \overset{t}{\dashrightarrow}_\ell}{p \overset{!o.t}{\dashrightarrow}_\ell}$$

Definition 1 (Non-interference, `NI`)**.** $p \in \mathrm{NI}(\overset{I}{\doteq}, \overset{O}{\doteq})$ iff for all ℓ, t, t', i and i':

$$i \overset{I}{\doteq}_\ell i' \;\Rightarrow\; \left(p \overset{t.?i.t'}{\dashrightarrow}_\ell \Rightarrow p \overset{t.?i'.t'}{\dashrightarrow}_\ell\right),$$
$$i \overset{I}{\doteq}_\ell \bullet \;\Rightarrow\; \left(p \overset{t.t'}{\dashrightarrow}_\ell \Leftrightarrow p \overset{t.?i.t'}{\dashrightarrow}_\ell\right).$$

The first clause (substitution) says that related inputs are interchangeable. The second says that an input the observer does not see may be inserted into a trace (insertion, $\Rightarrow$) or deleted from it (deletion, $\Leftarrow$) without the observer noticing. The set of traces visible at ℓ is thus unchanged by anything the observer at ℓ may not see.

C. Equivalences

A composite process gets its equivalences from those of its parts (Theorem 1). The constructors that combine or gate processes, `par`, `swi`, `maybe` and `sta`, each induce a constructor on equivalences; the others pass equivalences through. So the equivalences of the model in Section III are terms built from a few constructors over a few base equivalences, chiefly the two below.

We say that $\doteq$ *hides* x at ℓ when $x \doteq_\ell \bullet$. The *public* equivalence $\overset{\mathsf{L}}{\doteq}$ (`public_equiv`) hides nothing: at every level, $x \doteq_\ell y$ iff $x = y$. The *private* equivalence $\overset{\mathsf{H}}{\doteq}$ (`private_equiv`) hides everything from the attacker: at L every value is unobservable, so all values are related, and at H it is the public one. The superscript is the lowest level that sees the values. On the handler output $\{\texttt{Notify}, \texttt{Nothing}\}$, for example,

public: $\texttt{Notify} \not\doteq_{\mathsf{L}} \texttt{Nothing}$, $\texttt{Notify} \not\doteq_{\mathsf{L}} \bullet$,
private: $\texttt{Notify} \doteq_{\mathsf{L}} \texttt{Nothing} \doteq_{\mathsf{L}} \bullet$, $\texttt{Notify} \not\doteq_{\mathsf{H}} \texttt{Nothing}$.

The model gives the timer handler's output the public equivalence and the disk handler's the private one: the attacker learns when the timer handler finishes, but not when the disk handler does.

The other equivalences are built from these by the following formers.

Definition 2 (Equivalence formers)**.** Let $\overset{X}{\doteq}$ and $\overset{Y}{\doteq}$ be equivalences on X and Y. At each level ℓ, each former below relates and hides values as stated, and also relates any two values it hides.

- *Pairs* (`eqpair`, `eqpair_R`, `eqpair_LR`). $\overset{X}{\doteq} \times \overset{Y}{\doteq}$, $\overset{X}{\doteq} \times_R \overset{Y}{\doteq}$ and $\overset{X}{\doteq} \times_{LR} \overset{Y}{\doteq}$ relate (x, y) and (x', y') when $x \overset{X}{\doteq}_\ell x'$ and $y \overset{Y}{\doteq}_\ell y'$. The first hides no pair, the second hides (x, y) when $\overset{Y}{\doteq}$ hides y, and the third when $\overset{X}{\doteq}$ hides x and $\overset{Y}{\doteq}$ hides y.
- *Sums* (`eqsum_LR`). $\overset{X}{\doteq} + \overset{Y}{\doteq}$ relates $\mathsf{inl}\ x$ and $\mathsf{inl}\ x'$ when $x \overset{X}{\doteq}_\ell x'$, and $\mathsf{inr}\ y$ and $\mathsf{inr}\ y'$ when $y \overset{Y}{\doteq}_\ell y'$. It hides $\mathsf{inl}\ x$ when $\overset{X}{\doteq}$ hides x, and $\mathsf{inr}\ y$ when $\overset{Y}{\doteq}$ hides y.
- *Unit* (`unit_equiv`). For a set S of levels, closed upwards, 1^S relates the unit's element $\star$ to itself and hides it at the levels outside S. We write 1^ℓ for 1^S with S the levels at or above ℓ.

$\times$ is the output equivalence of `par`; $\times_R$ and $\times_{LR}$ appear where a constructor pairs an input with a state or a gate bit. Sums are our addition, and a conservative one: every constructor and theorem is parametric in its types, and none changes. An `inl` and an `inr` are related exactly when both are hidden; otherwise the tag is visible.

A process that does not run emits `None`, and under $\doteq + 1^S$ the levels in S see that it did not run; `Some v` and `None` are related at ℓ exactly when both are hidden. So 1^{L} lets every observer see `None`, 1^{H} hides it from the attacker, and $1^{\emptyset}$ hides it from everyone.

D. Compositional theorems

The calculus comes with one non-interference theorem per constructor. Each derives the equivalences of the composite from those of its parts, which is what gives the equivalences of a large model their shape. We write $\overset{I}{\doteq}$ and $\overset{O}{\doteq}$ for equivalences on a process's inputs and outputs, $\overset{V}{\doteq}$ for one on the state of a `sta`, and $\overset{B}{\doteq}$ for one on the gate bit of a `swi`; the theorems leave them open, and Section V instantiates each for the model. Their side conditions are stated with the following predicates on functions and levels:

$$
\begin{aligned}
f \in \mathrm{NI}(\overset{I}{\doteq}, \overset{O}{\doteq}) &\iff \forall \ell, i, i'.\ i \overset{I}{\doteq}_\ell i' \Rightarrow f\, i \overset{O}{\doteq}_\ell f\, i' \\
f \in \mathsf{PU}(\overset{I}{\doteq}, \overset{O}{\doteq}) &\iff \forall \ell, i.\ i \overset{I}{\doteq}_\ell \bullet \Rightarrow f\, i \overset{O}{\doteq}_\ell \bullet \\
f \in \mathrm{NI}(\overset{I}{\doteq}, \overset{V}{\doteq}, \overset{V}{\doteq}) &\iff \forall \ell, i, i', v, v'.\ i \overset{I}{\doteq}_\ell i' \wedge v \overset{V}{\doteq}_\ell v' \\
&\qquad \Rightarrow f\, i\, v \overset{V}{\doteq}_\ell f\, i'\, v' \\
f \in \mathsf{PE}(\overset{I}{\doteq}, \overset{V}{\doteq}) &\iff \forall \ell, i, v.\ i \overset{I}{\doteq}_\ell \bullet \Rightarrow f\, i\, v \overset{V}{\doteq}_\ell v \\
\ell \in \mathsf{A}(v, \doteq) &\iff \forall \hat{v}.\ v \doteq_\ell \hat{v} \Rightarrow v = \hat{v} \\
\ell \in \mathsf{O}(p, \doteq) &\iff \text{every output of } p \text{ at } \ell \text{ is } \doteq_\ell \bullet
\end{aligned}
$$

A function is non-interfering if it maps related values to related values (`f_NI`, `fv_NI`), unobservable-preserving if it maps unobservable values to unobservable ones (`f_PU`), and equivalence-preserving if an unobservable input does not visibly move the state (`f_EP`). An observer is *aware* of v (`aware`) if it can tell v from every other value, and *oblivious* to p (`oblivious`) if it sees none of p's outputs. For example, every level is aware of `true` under the public equivalence on booleans, but L is not under the private one, where `true` and `false` are related. And L is oblivious to a process all of whose outputs it relates to $\bullet$, such as a slot whose outputs, `None` included, are classified by $\doteq^{\mathsf{H}} + 1^{\mathsf{H}}$ (Section V-D).

Theorem 1 (Composition [4])**.** *For all processes, functions and equivalences of matching types:*

1) *(`out_NI`)* $\mathsf{out}\ o \in \mathrm{NI}(\overset{I}{\doteq}, \overset{O}{\doteq})$.
2) *(`map_NI`) If* $p \in \mathrm{NI}(\overset{I'}{\doteq}, \overset{O}{\doteq})$, $f \in \mathrm{NI}(\overset{I}{\doteq}, \overset{I'}{\doteq}) \cap \mathsf{PU}(\overset{I}{\doteq}, \overset{I'}{\doteq})$ *and* $g \in \mathrm{NI}(\overset{O}{\doteq}, \overset{O'}{\doteq})$, *then* $\mathsf{map}\ f\ g\ p \in \mathrm{NI}(\overset{I}{\doteq}, \overset{O'}{\doteq})$.
3) *(`sta_NI`) If* $p \in \mathrm{NI}(\overset{V}{\doteq} \times_R \overset{I}{\doteq}, \overset{O}{\doteq})$, $f \in \mathrm{NI}(\overset{I}{\doteq}, \overset{V}{\doteq}, \overset{V}{\doteq}) \cap \mathsf{PE}(\overset{I}{\doteq}, \overset{V}{\doteq})$ *and* $g \in \mathrm{NI}(\overset{O}{\doteq}, \overset{V}{\doteq}, \overset{V}{\doteq})$, *then* $\mathsf{sta}\ f\ g\ v\ p \in \mathrm{NI}(\overset{I}{\doteq}, \overset{V}{\doteq} \times \overset{O}{\doteq})$.
4) *(`swi_NI`) If* $p \in \mathrm{NI}(\overset{I}{\doteq}, \overset{B}{\doteq} \times_{LR} \overset{O}{\doteq})$ *and for every* ℓ, $\ell \in \mathsf{A}(\mathsf{true}, \overset{B}{\doteq}) \cup \mathsf{O}(\mathsf{swi}\ b\ p, \overset{O}{\doteq} + 1^{\mathsf{A}(\mathsf{true}, \overset{B}{\doteq})})$, *then* $\mathsf{swi}\ b\ p \in \mathrm{NI}(\overset{B}{\doteq} \times_{LR} \overset{I}{\doteq}, \overset{O}{\doteq} + 1^{\mathsf{A}(\mathsf{true}, \overset{B}{\doteq})})$.
5) *(`par_NI`) If* $p_1 \in \mathrm{NI}(\overset{I}{\doteq}, \overset{O_1}{\doteq})$ *and* $p_2 \in \mathrm{NI}(\overset{I}{\doteq}, \overset{O_2}{\doteq})$, *then* $\mathsf{par}\ p_1\ p_2 \in \mathrm{NI}(\overset{I}{\doteq}, \overset{O_1}{\doteq} \times \overset{O_2}{\doteq})$.
6) *(`loop_NI`) If* $p \in \mathrm{NI}(\overset{I}{\doteq}, \overset{I}{\doteq})$, *then* $\mathsf{loop}\ p \in \mathrm{NI}(\overset{I}{\doteq}, \overset{I}{\doteq})$.
7) *(`maybe_NI`) If* $p \in \mathrm{NI}(\overset{I}{\doteq}, \overset{O}{\doteq})$, *then* $\mathsf{maybe}\ p \in \mathrm{NI}(\overset{I}{\doteq} + 1^{\emptyset}, \overset{O}{\doteq})$.

Two of these carry substantial side conditions: the state update of `sta` must be non-interfering and equivalence-preserving, and `swi` requires every observer to be aware of the gate or oblivious to the gated process. These are the two obligations the proof of Section V has to discharge for the model.

III. An Operating System as One Process

[TODO: do a pass here, consider the handler just added] Section II gave us a calculus of processes, equivalences that say what an observer at each level sees, and one composition theorem per constructor. We now use the calculus to write an operating system with interrupt handling as a single process, following the example of Fig. 1: the secret is whether a disk interrupt occurred. We first introduce the processes the system runs and the pool in which they run (Section III-A), and what the system receives and emits (Section III-B). We then describe the state (Section III-C) and how each event updates it, ending in the choice of who runs next (Section III-D). The resulting

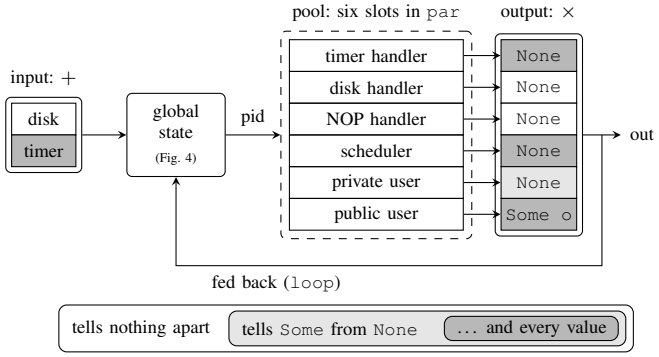


Fig. 3. The model. An interrupt only updates the global state. The state's pid selects one slot, which takes a step and emits `Some o`; every other slot emits `None`. Because the slots are composed with `par`, their outputs form a product, a six-tuple, which is emitted and fed back to update the state. The input is a sum of the two interrupts. Each position of the output, and each alternative of the input, is shaded by which of its values the attacker can tell apart (key below).

generic model leaves two parts open, which together form the *interrupt policy* (Section III-E); Section IV fills them twice.

A. The processes and the pool

The system runs six processes: three interrupt handlers, a scheduler, and two user processes. The timer, disk and NOP handlers are copies of one process, a counter:

```
ir_handler runtime :=
  map id (fun o => if o.1 == 0 then Notify
                   else Nothing)
    (sta (fun _ v => v)                    (* input
    *)
         (fun _ v => v.+1 %% runtime)      (* output
     *)
         0 (out tt))
```

The state cell holds a step count, which no input changes and each step advances modulo `runtime`. So a handler run takes exactly `runtime` steps, and on its last step the handler emits `Notify` to announce completion, and `Nothing` on the others. The public user process issues requests (`Get`) whose results the attacker sees. The private user process takes the disk handler's output as input, and emits `Sys` when it has been notified of a disk event and `Idle` otherwise. [NOTE: The private user process takes an input so that the effect of the disk handler's run is visible in user space: it is the legitimate consumer of the disk event.] When the scheduler runs, it proposes which user process runs next.

The six processes run side by side in a *pool* (Fig. 3), and only one of them takes a step at a time. Each sits behind a switch, controlled by the current process identifier (pid): a pid names one slot, whose process takes one step, while every other slot emits `None` without moving. Each slot is a `swi` whose input bit says whether the pid names it, which opens its switch; the slot tags each output with `true`, which closes the switch again, so a selected process takes exactly one step (Fig. 2). The pid is held in a global state, which a state cell and a feedback loop thread around the pool (`reactive_system`, Section III-E).

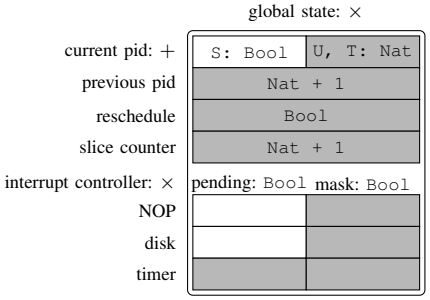


Fig. 4. The global state of Fig. 3, field by field, laid out and shaded as there (Section V-A). The current pid is a sum: its `inl` side holds a secret handler (an S step), a `Bool` saying which one, and its `inr` side the number of any other process (U or T). Its equivalence is $\doteq^{\mathsf{Ho}} + \doteq^{\mathsf{L}}$ (Definition 4), so the tag is public: whether a secret handler runs is public; which one runs is private. So are the pending bits of the NOP and disk handlers. Every other field, `None` included, is public.

B. Inputs and outputs

The system receives interrupts. Its input type `T_in` is a sum, like the current pid of Section III-C: `inl` is the disk interrupt and `inr` the timer interrupt. An interrupt goes to the state (Section III-C), where it marks itself pending in the interrupt controller.

By the rule for `par` (Fig. 2), the pool emits the outputs of all its slots together. So the system emits a six-tuple whose positions have types $A + 1$, with `Some` where the slot ran and `None` elsewhere, the output type `T_out`, with exactly one `Some` on each step. The output therefore records not only what was computed but also *who ran*: the schedule is part of what the system emits. The steps it records are of three kinds, which the rest of the paper keeps apart: *U*, a user process or the scheduler; *T*, the timer handler; and *S*, a secret handler, disk or NOP.

Definition 3 (Input and output equivalences)**.** The input equivalence $\mathcal{I}$ (`in_equiv`) and the output equivalence $\mathcal{O}$ (`out_equiv`) are

$$\begin{aligned}\mathcal{I} &\triangleq \doteq^{\mathsf{H}} + \doteq^{\mathsf{L}},\\ \mathcal{O} &\triangleq (\doteq^{\mathsf{L}} + 1^{\mathsf{L}})\\ &\quad \times (\doteq^{\mathsf{H}} + 1^{\mathsf{L}})\\ &\quad \times (\doteq^{\mathsf{L}} + 1^{\mathsf{L}})\\ &\quad \times (\doteq^{\mathsf{H}} + 1^{\mathsf{H}})\\ &\quad \times (\doteq^{\mathsf{H}} + 1^{\mathsf{H}})\\ &\quad \times (\doteq^{\mathsf{L}} + 1^{\mathsf{L}}),\end{aligned}$$

one factor per slot, in the order public user, private user, scheduler, NOP, disk and timer handler, with $\times$ associating to the right, as the output nests.

The disk interrupt is thus hidden from the attacker, and the timer interrupt is public. Of each output the attacker sees whether the slot ran and its value on the dark slots of Fig. 3, only whether it ran on the light slot, and nothing on the white ones. So the attacker sees the U and T steps, and of an S step

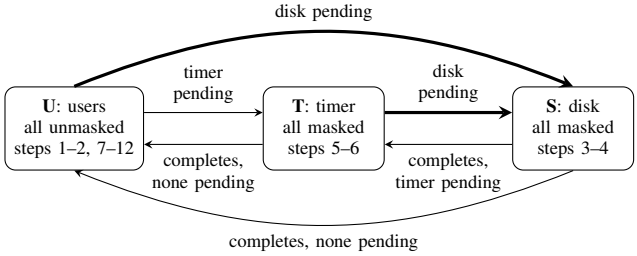


Fig. 5. The ordinary policy over three kinds of step: U, the users and the scheduler; T, the timer handler; S, a secret handler. Each state gives the masks and the steps of the run with the disk interrupt in Fig. 6; without it, S is never entered. Bold arrows are triggered by the disk interrupt: it decides when S is entered. Every completion arrow is taken by `immediate_service`, which reads the completing handler's output, the disk's included.

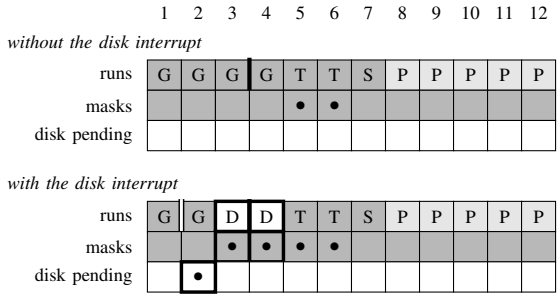


Fig. 6. The ordinary policy on the concrete system of Section IV, without and with a disk interrupt: who runs at each step (G: public user, P: private user, S: scheduler, T, D: timer and disk handler), and the state in which the step takes place (•: set). Shading as in Fig. 3: public cells shaded. A solid bar marks the timer interrupt, a hollow bar the disk interrupt; framed cells differ between the two runs. The disk handler takes steps 3 and 4 from the public user and holds the masks set meanwhile, so the runs differ in public cells. Both runs are proved traces, and the states are computed by the model's own transition (`models.v`, Section 11).

only that it happened: no other slot ran. Section V-D shows that the proof forces this shape. Each output, like each input, then updates the state.

The *user view* keeps only the two user slots, as one value of type `(Opub + Opriv) + 1`:

```
parse_output o :=
  match o with
  | (Some v, _)          => Some (inl v)
  | (None, (Some v, _)) => Some (inr v)
  | _                    => None
  end
userview m := map id parse_output m
```

Definition 4 (Visible equivalence)**.** The visible equivalence $\doteq^{\circ}$ (`eqvisible`) of an equivalence $\doteq$ relates what $\doteq$ relates and hides nothing:

$$x \doteq^{\circ}_{\ell} y \iff x \doteq_{\ell} y, \qquad x \doteq^{\circ}_{\ell} \bullet \text{ never.}$$

At L a private-user output and `None` are both hidden, so a plain sum would relate them and hide whether the private user ran. Under a sum, $^{\circ}$ keeps the tag visible, so the user view applies it to the private user's equivalence.

Definition 5 (User-view equivalence)**.** The user-view equivalence $\mathcal{U}$ (`out_equiv_userview`) is

$$\mathcal{U} \triangleq (\doteq^{\mathsf{L}} + \doteq^{\mathsf{H}\circ}) + 1^{\emptyset},$$

writing $\doteq^{\mathsf{H}\circ}$ for $(\doteq^{\mathsf{H}})^{\circ}$.

So the attacker still sees when the private user runs. The user view thus erases only public information: the scheduler and the timer handler, and which of them, or S, ran.

C. The state

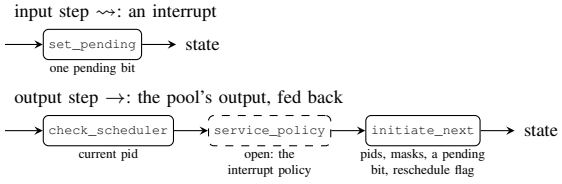
The global state (`state_type`, Fig. 4) is the box of Fig. 3 opened up: a nest of pairs holding two pids, two flags and the interrupt controller.

- The *current pid* names the slot that runs next. The two secret handlers, the S steps, sit on the `inl` side, `inl true` for the disk handler and `inl false` for the NOP handler, and every other process on the `inr` side, numbered from 0: timer handler, scheduler, private user, public user. The system starts in the public user (`initial_pid`).
- The *previous pid* is the user process a handler interrupted, to which control returns when the handler finishes.
- The *reschedule flag* `re_sched` sends control to the scheduler instead, when a handler finishes.
- The *slice counter* `slice_count` counts the steps left in the current time slice, for the sliced policy (Section IV-B); the ordinary policy leaves it `None`.
- The *interrupt controller* `ic` holds a pending bit and a mask bit for each handler. Its entries are indexed by `irq`, of type $\mathit{Interrupt} + 1$: `timer_irq` and `disk_irq` for the two interrupts, and `nop_irq`, which is `None`, for the NOP handler. No interrupt raises the NOP entry; the sliced policy sets it pending itself, to use the NOP handler as filler.

The initial state is not fixed here: it is part of the interrupt policy (Section III-E).

D. Who runs next

The state changes on both kinds of step of Section II, through `state_step`; under each stage are the fields of Fig. 4 it writes:



On an input step, `set_pending` sets the arriving interrupt's pending bit and nothing else happens: the pool idles and nothing is emitted. On an output step, the pool's output is fed back to the state, where three stages run in turn. `check_scheduler` adopts the pid the scheduler proposed, if the scheduler ran. The next stage, `service_policy`, is left open here: it is half of the interrupt policy (Section III-E),

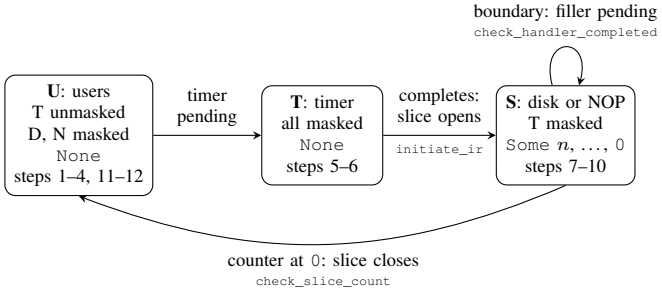


Fig. 7. The sliced policy over the states of Fig. 5, with the slice counter in each state. No arrow is bold: S is entered only when the timer handler completes, and left only when the counter, loaded with $runs \cdot runtime$, reaches zero. Inside S the disk handler, if pending, takes a run the NOP handler would otherwise fill; that choice is all the secret decides. Each arrow is labelled with the stage of the service policy that takes it. Steps are those of both runs in Fig. 8.

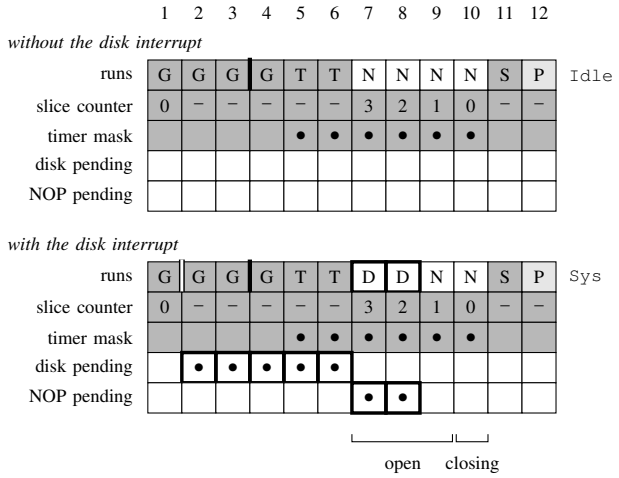


Fig. 8. The sliced policy on the same inputs, laid out as Fig. 6; N: NOP handler. The disk and NOP masks are set between all steps and are not shown. The framed cells, where the runs differ, are all private: the disk pending bit until the slice opens, then which handler fills the first run and the NOP pending bit. On the right, the private user's output at step 12; the braces mark the slice, its last step being the closing one (Fig. 7).

and Section IV fills it twice. Last, `initiate_next` computes the next pid. (In the code the state receives both kinds of event as one sum, interrupts as `inl` and fed-back outputs as `inr` (Section III-E), and `step_sum` dispatches on the tag.)

A handler can be started when its controller entry is pending and not masked; if several can, the timer goes first, then the disk, then NOP. Handlers cannot be interrupted: starting one sets all masks, and the service policy unsets them when the handler finishes. `initiate_next` chooses by four rules, tried in order:

1) if a handler is running (all masks are set), it continues;
2) otherwise, if a handler can be started, it starts;
3) otherwise, if a handler has just finished, control returns to the user process it interrupted, or to the scheduler if a reschedule was requested;
4) otherwise, the current process continues.

In the code, with the rules marked:

```
initiate_next restore_invariant v =
  if masks_set v then v                (* rule 1 *)
  else let v := restore_invariant v in
    if first_ready v is Some ir
    then initiate_handler ir v         (* rule 2 *)
    else if is_handler_pid v           (* rule 3 *)
    then if get_re_sched v
         then initiate_scheduler v
         else initiate_prev_pid v
    else v                             (* rule 4 *)
```

`first_ready` picks the first handler that can be started, in the order above. `initiate_handler` saves the current pid as the previous pid, unless it is a secret handler, points the current pid at the handler, sets all masks and clears the handler's pending bit. `initiate_scheduler` hands control to the scheduler and clears `re_sched`; `initiate_prev_pid` returns to the previous pid. Between rules 1 and 2, `initiate_next` applies `restore_invariant`, a hook used only by the proof (Section III-E).

Rule 2 is the source of the leak of Fig. 1, as Section IV-A shows step by step.

E. The generic model

Together, these parts form the generic model, the pool inside the global state:

```
model runtime init_policy service_policy
      restore_invariant p_pub p_priv p_sched =
  reactive_system init_policy
    (state_step service_policy restore_invariant)
    no_output
    (pool runtime p_pub p_priv p_sched)
    pool_input
  : Proc T_in T_out
```

Its first argument, `runtime`, is the handler length, and its last three are the user processes and the scheduler; the user processes' output types, also arguments, are left implicit. The wrapper `reactive_system` is the loop of Fig. 3, built from the constructors of Fig. 2:

```
reactive_system state state_update
                no_output p pool_input =
  map inl (inr_or_def no_output)
    (loop (map id snd
      (sta state_update (fun _ v => v) state
        (map pool_input inr (maybe p)))))
```

Inside the loop an event is a sum: the outer `map` tags an arriving interrupt `inl`, and the inner `map` tags each output of the pool `inr`, which `loop` feeds back. The `sta` cell is the global state, updated by `state_update` on every event and left alone on an output. `pool_input` gives the pool the new current pid, and the private user the disk handler's output, after a fed-back output, and `None` after an interrupt, on which `maybe` leaves the pool unchanged. On the way out, `map id snd` drops the state that `sta` emits with each output, and `inr_or_def` strips the `inr` tag. Its default `no_output`, the all-`None` tuple, only makes it total: every output is tagged `inr`.

Apart from the processes and the handler length, everything in Sections III-A to III-D is fixed, except two parts, marked by the suffix `_policy`:

- `init_policy`, the initial state, a value of the type of Section III-C. Its masks decide which interrupts can be serviced before anything has happened.
- `service_policy`, the open stage of Section III-D. It is the stage that reopens masks, and so decides when the next handler becomes eligible.

Together they decide *when a handler may run*, and we call them the *interrupt policy*. The remaining argument, `restore_invariant`, is not part of it. It writes into the state an invariant that the policy already maintains, for the benefit of the proof (Section V-C); it never changes a state the model reaches (Theorem 5), and it is the identity where the proof needs no help. Section IV fills the two parts of the interrupt policy twice.

The model is parametric in more than the policy. The scheduler, the two user processes and their output types are arguments, as is the handler length; the sliced policy adds the number of handler runs per slice. Theorem 4 holds for all of them, provided each slot process is non-interfering at its own classification: the mechanism cannot make a leaking process secure, only add no leak of its own. The handler length and slice size need no condition, because a slice is defined as $runs \cdot runtime$ steps and so ends on a handler boundary. This generality rests on one property of the state update: it reads only *whether* a user slot emitted, never what, so no user behaviour reaches the schedule. What is fixed is the mechanism under study: the three handlers, and the six slots in the order the state update reads them.

IV. Two Interrupt Policies

We now instantiate the model with two interrupt policies: `model_immediate`, which leaks (Section IV-A), and `model_sliced`, which does not (Section IV-B). Both are exercised on the same concrete system. Its public user process emits the request `GetRequest` (`Get`) whenever it runs, and its private user process emits `Syscall` (`Sys`) when the disk handler has notified it and `Idle` otherwise. The scheduler alternates between the two user processes. Handlers take $runtime = 2$ steps.

Recall the three kinds of step (Section III-B): U, a user process or the scheduler; T, the timer handler; S, a secret handler. The attacker sees when U and T steps happen, and of an S step only that it happened, never which secret handler ran. It thus learns when the system enters S and how long it stays there, so for the attacker to learn nothing, both must be fixed by public events; what runs inside S may depend on the secret. Figures 5 and 7 draw the two policies over U, T and S, and Figs. 6 and 8 run them on the same inputs.

A. Ordinary interrupt handling, and the leak

`model_immediate` services an interrupt as soon as no handler is running (Fig. 5). A pending timer interrupt takes the system from U to T, and a pending disk interrupt takes it to S, from U or from T, at whatever step it arrives. Each completion unmasks everything and moves on to the next pending handler, or back to U. The two bold arrows into S are the leak: the secret decides when S is entered.

The initial state

```
immediate_init :=
  ((inr 3, None),        (* public user *)
   (false, (None,         (* slice counter *)
    ((false,false),       (* NOP *)
     ((false,false),      (* disk *)
      (false,false))))))  (* timer *)
```

starts in the public user with the slice counter disabled (fields as in Section III-C). The interrupt controller holds one (pending, mask) pair per handler, and all six bits start clear. The service policy is a single stage, which reacts to any handler finishing:

```
immediate_service o v :=
  if is_ir_out_done o is Some ir then
    let v := unset_masks v in
    if ir is timer_irq then update_re_sched v true
    else v
  else v
```

`is_ir_out_done` names the handler, if any, that emits `Notify` in the output `o`. The service policy then unmasks everything, and after the timer handler also requests a reschedule. It thus reads the output of whichever handler completes, the disk handler's included. `restore_invariant` is the identity.

The run with the disk interrupt in Fig. 6 shows the policy at work. The pid of each step is computed at the output of the step before (Section III-D). The disk interrupt arrives after step 1 and only sets the disk pending bit, so step 2 still belongs to the public user. At the output of step 2, `initiate_next` finds the disk handler pending and unmasked and starts it, setting all masks: U to S. The timer interrupt that arrives meanwhile stays pending until the disk handler emits `Notify` at step 4: S to T. The timer handler's completion at step 6 requests a reschedule: T to U, where the scheduler hands step 8 to the private user. Without the disk interrupt the system never enters S. Its two steps are taken from the public user, and the masks, which are public, are set at steps 3 and 4 in one run only.

Theorem 2 (`model_immediate_not_NI`)**.** *For the concrete system,*

$$\mathsf{model_immediate_concrete} \notin \textsc{ni}(\mathcal{I}, \mathcal{O}).$$

Proof sketch. The counterexample is the shortest trace that shows the gap: the disk interrupt arrives before step 1, not after it as in Fig. 6. Let p be the model and o_G the output in which only the public slot is set, to `Get`. The model performs $p \xrightarrow{!o_G.!o_G} \mathsf{L}$. The disk interrupt is unobservable at L, so insertion demands $p \xrightarrow{?\mathsf{disk}.!o_G.!o_G} \mathsf{L}$. Reduction is deterministic (Section II): p has exactly one input step on disk and every process exactly one output step, so we only need to follow that run. The input step $p \overset{\mathsf{disk}}{\rightsquigarrow} p_1$ sets the disk pending bit and leaves the pid at its initial value, the public user. So the output step $p_1 \xrightarrow{o_1} p_2$ emits $o_1 = o_G$, and at its end `initiate_next`

finds the disk handler pending and unmasked and sets the pid to the disk handler: U to S. The next output step $p_2 \xrightarrow{o_2} p_3$ is the disk handler's, so o_2 has `None` in the public slot, and $o_2 \not\mathcal{O}_{\mathsf{L}} o_G$. □

The leak is visible in the user view as well:

Theorem 3 (`model_immediate_userview_not_NI`)**.** *For the concrete system,*

$$\text{userview model_immediate_concrete} \notin \text{NI}(\mathcal{I}, \mathcal{U}).$$

The missing `Get` is a public-user output, which `parse_output` keeps, so the counterexample and the proof are those of Theorem 2, with every output passed through `parse_output`.

B. Sliced interrupt handling

The sliced policy removes the bold arrows. It runs the secret handlers only in *time slices*: S is entered only when the timer handler completes, and the system stays there for a fixed number of steps (Fig. 7). A slice holds exactly *runs* handler runs ($runs = 2$ in the concrete system), and a run that no disk interrupt claims is taken by the NOP handler, so a slice is the same whether or not a disk interrupt is waiting. Each part of the policy below removes an arrow of Fig. 5 or fixes how long S lasts.

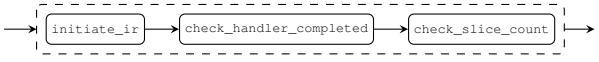
The slice counter tracks the slice. Outside it, in U and T, the counter is `None`. In S, `Some n` counts the n steps left, and `Some 0` marks the last step, at whose output the slice closes. Closing must happen once. With a plain number, `0` could not tell a closing slice from a closed one, and closing on every step outside a slice would unmask the timer while its own handler runs.

The initial state masks the disk and NOP handlers, so no disk interrupt can take the system from U to S, and leaves the timer unmasked:

```
sliced_init :=
  ((inr 3, None),          (* public user *)
   (false, (Some 0,        (* slice counter *)
    ((false,true),         (* NOP *)
     ((false,true),        (* disk *)
      (false,false))))))   (* timer *)
```

Its counter, `Some 0`, is an empty slice that closes at the first output and changes nothing else.

The service policy fills the open stage of Section III-D with three stages, `sliced_service`, which run on each output in this order:



The first stage is the ordinary policy's single stage restricted to the timer handler, whose output is public. It keeps the reschedule and, instead of unmasking, opens the slice, the only way from T to S:

```
initiate_ir runtime runs o v :=
  if tI_out o is Some Notify then
    update_re_sched
      (update_slice_count v
         (Some (runs * runtime)))
      true
  else v
```

When the timer handler emits `Notify`, it loads the counter with $runs \cdot runtime$, the number of steps that *runs* handler runs take, and requests a reschedule, which returns control to U, through the scheduler, once the slice is over. The second stage acts at the boundaries between runs:

```
check_handler_completed runtime v :=
  if handler_completed runtime
       (get_slice_count v) then
    update_ir_pending (set_tI (unset_masks v))
      nop_irq true
  else v
```

`handler_completed` holds when the counter is a nonzero multiple of *runtime*: when the slice opens and after each completed run inside it. Unmasking has moved here from the first stage: it happens at boundaries computed from the public counter, not on completion signals. Since every run takes *runtime* steps, the two fall on the same steps; only the reason has changed. There the masks reopen, but the timer is masked again, so it cannot leave S early, and the NOP handler becomes pending. That last step is the *filler*: a handler is always ready at a boundary, so S is entered and every run of the slice takes place whether or not a disk interrupt is waiting. A pending disk handler takes priority; which of the two runs is all the secret decides. The third stage, `check_slice_count`, ticks the counter, and at `Some 0` closes the slice: it masks the disk and NOP handlers, so U cannot lead to S, unmasks the timer and disables the counter.[1] `restore_invariant` only matters to the proof (Section V-C).

V. The Security Proof

This section proves that the sliced policy is non-interfering, with $\mathcal{I}$ and $\mathcal{O}$ of Definition 3:

Theorem 4 (`model_sliced_NI`)**.** *If the scheduler and the public user process are non-interfering at* $\dot{=}^{\mathsf{L}}$ *on input and output, and the private user process is non-interfering at* $\dot{=}^{\mathsf{H}}$ *on input and output, then*

$$\text{model_sliced } runtime\ runs\ p_{pub}\ p_{priv}\ p_{sched} \in \text{NI}(\mathcal{I}, \mathcal{O}).$$

[TODO: Section-wide: write every step the way the corollary's proof now reads: name the process term, the item of Theorem 1 with its instance (p, f, g, the equivalences), list its side conditions, and say why each holds. Several steps below give only the gist and leave the instance implicit.]

At the attacker's level L the conclusion reads as follows. A disk interrupt is unobservable at L (Section III-B), so two input streams that differ only in disk interrupts must give outputs that $\mathcal{O}$ relates.

The result carries over to the user view (Section III-B):

Corollary 1 (`model_sliced_userview_NI`)**.** *Under the same hypotheses, the user view of the sliced model,* `model_sliced_userview`*, is in* $\text{NI}(\mathcal{I},\mathcal{U})$.

[1] With one-step handlers the last run of each slice is lost (Section V-C); non-interference is unaffected.

[TODO: "adds one layer outside the model" mixes the term (a map around model_sliced) with the proof (one more use of map_NI). Say it as a proof: put this in a proof environment, starting "The user view is the process . . . ".] Its proof adds one layer outside the model. The user view is the process

$$\mathsf{map\ id\ parse_output\ model_sliced},$$

so Theorem 1(2) (`map_NI`) applies with $p \in \text{NI}(\mathcal{I}, \mathcal{O})$ by Theorem 4, $f = \mathsf{id}$ and $g = \mathsf{parse_output}$. Its side conditions are $\mathsf{id} \in \text{NI}(\mathcal{I}, \mathcal{I}) \cap \mathsf{PU}(\mathcal{I}, \mathcal{I})$, which holds trivially, and $g \in \text{NI}(\mathcal{O}, \mathcal{U})$, that is,

$$o \overset{\mathcal{O}}{\doteq}_\ell o' \;\Rightarrow\; g\,o \overset{\mathcal{U}}{\doteq}_\ell g\,o'$$

for every level ℓ and outputs o, o'. This holds because `parse_output` reads only the two user slots. If $o \overset{\mathcal{O}}{\doteq}_\ell o'$, then o and o' agree on which user slot, if any, emitted, and on the public user's value, so $g\,o$ and $g\,o'$ agree on the tag and the public value; if neither user slot emitted, $g\,o$ and $g\,o'$ are both `None`.

[TODO: "from the outside in" is not explained. Say what it means: model_sliced is a term (the reactive_system listing of Section III-E); the proof applies one item of Theorem 1 per constructor, starting at the outermost and moving to its argument; each application turns the goal for the term into goals for its argument, at equivalences the item dictates, plus side conditions on its functions. Consider displaying the unfolded term once here, so that each bullet below can point at its constructor.] The proof applies Theorem 1 to `model_sliced` (Fig. 3) from the outside in. The theorem gives the outermost equivalences, $\mathcal{I}$ and $\mathcal{O}$, shaded in Fig. 3; the proof chooses those inside, $\overset{V}{\doteq}$ on the state (Fig. 4) and $\overset{B}{\doteq}$ on each gate bit (Section V-D).

[TODO: "five layers" is informal and the grouping is ours, not the term's: the term has four maps (the outer inl map, map id snd, the pool's input map, the gate maps) and the bullets do not say which map each refers to. Use the constructor order of the listing, and name each map.] The term has five layers. For each, from the outside in, we name its constructors, the items of Theorem 1 that apply, and what they require:

- *Feedback* (`map`, `loop`; Theorem 1(2), Theorem 1(6)). The outer map tags each interrupt `inl`, and the loop feeds each output back tagged `inr`, so the state receives one sum of events. Theorem 1(6) asks for the process inside the loop to be in $\text{NI}(\overset{E}{\doteq}, \overset{E}{\doteq})$, where we write $\overset{E}{\doteq}$ for $\mathcal{I} + \mathcal{O}$, the two equivalences of Fig. 3 side by side. No output is unobservable, so it never relates an interrupt to an output, and a disk interrupt is unobservable at L. [TODO: Inaccurate: the loop does not tag outputs inr, the inner map (map pool_input inr) does. The side conditions of map_NI for the outer map (inl $\in$ NI $\cap$ PU, inr_or_def $\in$ NI) and for map id snd are not stated.]
- *The state* (`sta`; Theorem 1(3)). The state update f must satisfy $f \in \text{NI}(\overset{E}{\doteq}, \overset{V}{\doteq}, \overset{V}{\doteq}) \cap \mathsf{PE}(\overset{E}{\doteq}, \overset{V}{\doteq})$. It decides whether a secret handler runs, and it is the hard obligation (Sections V-A to V-C). [TODO: f here is sta's state update, but f was map's input function one bullet up and in the corollary. sta_NI also requires the output update (g, the identity here) to be in $\text{NI}(\overset{O}{\doteq}, \overset{V}{\doteq}, \overset{V}{\doteq})$, and gives the inner process inputs at $\overset{V}{\doteq} \times_R \overset{E}{\doteq}$; neither is mentioned.]
- *The pool* (`map`, `maybe`, `par`; Theorem 1(2), Theorem 1(7), Theorem 1(5)). The pool idles on interrupts (Section III-D): its input map sends each to `None`, on which `maybe` idles (Fig. 2), and every output to the current pid. `maybe` gives the pool's input the equivalence $\cdot + 1^{\emptyset}$, under which no level sees `None`, so the map is in PU. `par` splits the output equivalence into one component per slot. [TODO: Name the map (map pool_input inr) and the equivalence: "$\cdot + 1^{\emptyset}$" leaves the payload (pid $\times$ disk output) unnamed. State pool_input's side conditions exactly (NI and PU from $\overset{V}{\doteq} \times_R \overset{E}{\doteq}$). "par splits" reads backwards: par_NI composes the slots' equivalences into the product.]
- *The slots* (`map`, `swi`; Theorem 1(2), Theorem 1(4)). Each gate map must be in NI into $\overset{B}{\doteq}$, and each slot must satisfy the awareness-or-obliviousness condition of Theorem 1(4); this hides which secret handler runs (Section V-D).
- *The slot processes.* p_{sched}, p_{pub} and p_{priv} are in NI by the hypotheses of Theorem 4. Each handler is in NI by its definition (Section III-A): `out tt` by Theorem 1(1); the `sta` around it by Theorem 1(3), with a public count whose input update is the identity, so it is in PE; and the `map` by Theorem 1(2).

A. The state equivalence

[TODO: $\overset{V}{\doteq}$ is given only by the shading of Fig. 4 and the bullets below. Give it a definition like Definition 3 ($\overset{V}{\doteq} \triangleq \ldots$, one factor per field), then justify each factor. "The first condition asks for private fields, the second for public ones" is vague: say that PE is easier to meet the more fields are private, NI the more are public.] For the state, Theorem 1(3) asks for $f \in \mathsf{PE}(\overset{E}{\doteq}, \overset{V}{\doteq})$, so an unobservable event leaves the state related to itself, and $f \in \text{NI}(\overset{E}{\doteq}, \overset{V}{\doteq}, \overset{V}{\doteq})$, so related events take related states to related states (Section II-D). The first condition asks for private fields, the second for public ones. We make a field private only where a condition forces it, which gives $\overset{V}{\doteq}$, the shading of Fig. 4 (`state_type_equiv`):

- *Pending bits are private.* A disk interrupt is unobservable at L and sets the disk pending bit, so $f \in \mathsf{PE}(\overset{E}{\doteq}, \overset{V}{\doteq})$ makes that bit private. The NOP pending bit is private by $f \in \text{NI}(\overset{E}{\doteq}, \overset{V}{\doteq}, \overset{V}{\doteq})$: when a slice opens, the disk handler takes the run the NOP handler was queued for, so after that run the NOP handler is pending in one state and not in the other.

- *The current pid is* $\dot{=}^{\mathsf{H}\circ} + \dot{=}^{\mathsf{L}}$. Its `inl` side holds exactly the two secret handlers (Section III-C), and which of them runs may differ between the two states, so the value inside `inl` is private. The $^\circ$ makes no `inl` unobservable, so the sum never relates an `inl` to an `inr`, and *whether* a secret handler runs stays public.
- *Every other field is public:* the masks, the slice counter, the reschedule flag and the previous pid. Nothing forces them private, and `initiate_next` branches on all of them to compute the next pid, whose tag must be public: two states that disagreed on a mask, or on whether a slice is open, could enter S in one and not in the other.

The same conditions explain why `set_pending` sets one bit and stops: any decision taken on an input step, such as starting a handler or moving the counter, would change a public field, so everything an interrupt causes waits for a later output step.

B. The state update

$f \in \mathsf{PE}(\overset{E}{\dot{=}}, \overset{V}{\dot{=}})$ concerns unobservable events only, that is, disk interrupts, which `set_pending` turns into one private bit; it is immediate. The hard obligation is $f \in \mathrm{NI}(\overset{E}{\dot{=}}, \overset{V}{\dot{=}}, \overset{V}{\dot{=}})$. It splits along the two lanes of `state_step` (Section III-D), since $\overset{E}{\dot{=}}$ never relates an interrupt to an output, into

$$\mathsf{set_pending} \in \mathrm{NI}(\mathcal{I}, \overset{V}{\dot{=}}, \overset{V}{\dot{=}}) \quad \text{and}$$
$$s \in \mathrm{NI}(\mathcal{O}, \overset{V}{\dot{=}}, \overset{V}{\dot{=}})$$

[TODO: above, what is s?] [TODO: Answer to the above: say before the display that the output lane is the composite initiate_next ∘ service_policy ∘ check_scheduler (Section III-D), and that fv_NI_comp lets the proof show NI for each of the three stages s separately.] for each stage s of the output lane (`fv_NI_comp`). The first holds because `set_pending` writes a private bit. The output lane of the sliced policy has six stages, applied in this order:

1) `check_scheduler` reads the scheduler's output and writes the current pid from it;
2) `initiate_ir` reads the timer handler's output and, on `Notify`, loads the counter and sets the reschedule flag;
3) `check_handler_completed` reads the counter and, at a handler boundary, unmasks the disk and NOP handlers, masks the timer and sets the NOP pending bit;
4) `check_slice_count` reads the counter and ticks it, and at `Some 0` masks the disk and NOP handlers, unmasks the timer and disables the counter;
5) `restore_invariant` reads the counter and writes masks and the NOP pending bit to match it (Section V-C);
6) `initiate_next` reads the masks, the pending bits, the pid and the reschedule flag, and chooses the next pid by the rules of Section III-D.

The first five read only public fields of the state and public slots of the output, so on related arguments they write the same values, private bits included, and their obligations are routine. The service policy, stages 2 to 4, reads only the timer handler's output; this is what the restriction of Section IV-B buys. `initiate_next` is different: it reads the private pending bits. The timer's readiness is public, so both states agree on it. Otherwise, in each state the secret handler ready to start is the disk handler, the NOP handler, or none. Disk in one state and NOP in the other is harmless: both start an `inl` pid, whose value is private. A secret handler in one state and none in the other is fatal: one state enters S and the other continues a user process. [TODO: "about to choose" is informal; state the fact as a property of two states $v \overset{V}{\dot{=}}_{\mathsf{L}} v'$ that reach initiate_next, and give it a name the next two subsections can refer to.] So the stage needs one fact: two related states, about to choose, either both have a secret handler ready or neither has. The ordinary policy fails here and in the service policy. `immediate_service` unmasks when any handler completes, the disk handler included, so it writes public masks from a private output. And `initiate_next` meets the fatal case: a bold arrow of Fig. 5 taken in one run only, which is the counterexample of Theorem 2.

In the states the sliced policy reaches, the public counter gives the fact: while a slice is open the NOP handler is ready, and otherwise neither secret handler is. But $\mathrm{NI}(\mathcal{O}, \overset{V}{\dot{=}}, \overset{V}{\dot{=}})$ quantifies over all pairs of related states, reachable or not.

C. Restoring the invariant

The output lane is one function of the output and the state, the composite of the six stages of Section V-B. Proving it non-interfering in one piece would mean following all their branches at once. `fv_NI_comp` instead proves it stage by stage: if every stage takes related arguments to related states, so does their composite. The price is what the composite forgets between stages. Each stage's obligation is stated for every pair of related states, so all that passes from one stage to the next is that the two states are related. What an earlier stage establishes about its result is lost: that `check_handler_completed` has just made the NOP handler pending, or that `check_slice_count` has just masked the disk and NOP handlers. `initiate_next` must therefore be non-interfering on *every* pair of related states, and that fails. Two related states agree on the counter and the masks but may differ in the private pending bits; if the disk and NOP handlers are unmasked, the disk handler is pending in one state, and no secret handler in the other, one state enters S and the other does not.

What is forgotten is the link between the counter and the secret handlers' bits. The sliced policy maintains it on every state it reaches in which not all masks are set, that is, at every choice that rule 1 of `initiate_next` does not settle:

1) the NOP mask equals the disk mask;
2) inside an open slice, the NOP handler is pending and the timer masked;
3) outside a slice, the disk and NOP handlers are masked.

With these three conditions, whether a secret handler is ready depends only on the counter and the masks, which are public: inside a slice, exactly when the disk and NOP handlers are unmasked; outside it, never. So two related states that satisfy them agree on whether a secret handler is ready, which is the fact `initiate_next` needs.

Two ways of putting the conditions back fail. $\overset{V}{\doteq}$ cannot be restricted to the states that satisfy them, because a characterised equivalence relates every value to itself. And arguing over reachable states only would give up the per-constructor structure of the proof. Instead the model writes the conditions into the state: `initiate_next` applies `restore_invariant` just before it chooses (Section III-D). `restore_invariant` reads the counter and, in an open slice, sets the NOP pending bit and the timer mask, otherwise the disk and NOP masks, and in both cases copies the disk mask to the NOP mask. Every state it returns satisfies the three conditions, so the proof of the choice needs to consider only states in its image. It reads only public fields, so it is itself a routine stage, $\mathsf{restore_invariant} \in \mathrm{NI}(\overset{V}{\doteq}, \overset{V}{\doteq})$.

The proof of Theorem 4 needs nothing more. What remains is to show that the restriction costs nothing: `restore_invariant` is the identity wherever the model applies it, so `model_sliced` behaves exactly as the sliced policy without it. Write v_{es} for the state after the events es from the initial state of `model_sliced`, and $c_o(v)$ for the state that `initiate_next` receives on output o in state v, after `check_scheduler` and the service policy.

Theorem 5 (`restore_invariant_noop`)**.** *Let* $\mathit{runtime} \neq 1$. *For all events* es *and outputs* o*, if not all masks are set in* $c_o(v_{es})$*, then* $\mathsf{restore_invariant}\ (c_o(v_{es})) = c_o(v_{es})$.

`initiate_next` applies `restore_invariant` only when not all masks are set, so it never changes a state the model reaches. The events es are arbitrary, so this holds whatever the pool emits.

The proof of Theorem 5 is an invariant between steps (`sliced_inv`: the disk and NOP masks agree, are set outside a slice, and a handler is running inside one), preserved by every event. At the choice point a variant (`choice_inv`) also allows a handler boundary inside a slice, where the masks are open and the NOP handler is pending; on such states `restore_invariant` is the identity.

The hypothesis $\mathit{runtime} \neq 1$ marks where `restore_invariant` and the counter disagree: `restore_invariant` treats a closing slice, at `Some 0`, as closed. Normally no choice is made then, because the last run is still in progress. With one-step handlers the last boundary of a slice falls on that step, and `restore_invariant` masks the handler the boundary has just enabled. Non-interference is unaffected, but at $\mathit{runtime} = 1$ every slice loses its last handler run.

D. Gating a slot

Inside `sta`, `par` gives each slot its own component of the output equivalence, and each slot is a `swi` whose gate bit a `map` computes from the pid. Both obligations depend on the equivalence $\overset{B}{\doteq}$ on the gate bit, which the proof chooses per slot. [TODO: g is reused yet again (map's output function, parse_output, now the gate map). Give the gate map its own name. Also "Inside sta" and "par gives each slot its own component" describe the term, not the proof step; state the instance of map_NI and swi_NI per slot as in the corollary.] With g the gate map on pids, Theorem 1(2) and Theorem 1(4) require

$$g \in \mathrm{NI}(\overset{\mathsf{H}\circ}{\doteq} + \overset{\mathsf{L}}{\doteq}, \overset{B}{\doteq}),$$
$$\forall \ell.\ \ell \in \mathsf{A}(\mathsf{true}, \overset{B}{\doteq}) \ \cup\ \mathsf{O}(\mathsf{swi}\ b\ p,\ \overset{O}{\doteq} + 1^{\mathsf{A}(\mathsf{true}, \overset{B}{\doteq})}) :$$

related pids go to related bits, and every observer is aware of the gate or oblivious to the slot. (The other condition of Theorem 1(2), $g \in \mathsf{PU}$, holds trivially: no input of the gate map is unobservable.) Awareness means a bit related to `true` is `true` and observable, so the observer can trust the gate; obliviousness means it sees none of the slot's outputs. At L the two secret pids are related, and the slots fall into two groups (Fig. 9).

Every other slot shows whether. Both secret pids map to `false`, so related pids go to equal bits, and we take for $\overset{B}{\doteq}$ the equivalence $\overset{F}{\doteq}$ of Fig. 9 (`false_equiv`): $\doteq^{\mathsf{H}} + \doteq^{\mathsf{L}}$ on a bit read as a sum, `false` on the private side (`false_equiv_sum_rel`). So `false` is unobservable at L and `true` is public. Every level is aware of `true`, so $\mathsf{A}(\mathsf{true}, \overset{F}{\doteq}) = \{\mathsf{L}, \mathsf{H}\}$ and the output wrapper is [TODO: "output wrapper" is not a term the paper defines: it is the output equivalence $\overset{O}{\doteq} + 1^{\mathsf{A}(\mathsf{true}, \overset{B}{\doteq})}$ that swi_NI gives. Say that, here and in the next paragraph.] $\cdot + 1^{\mathsf{L}}$: the attacker tells `Some` from `None`, so it sees whether the slot ran. While a secret handler runs, all these slots emit `None`; this is how the attacker sees that the system is in S. The private user's slot is among them; its secrecy is carried by its private payload.

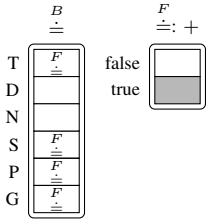


Fig. 9. Gate bits: $\overset{B}{\doteq}$ per slot, shaded as in Fig. 3.

The disk and NOP slots hide which. One secret pid maps to `true` and the other to `false`, so $g \in \mathrm{NI}$ needs `true` and `false` related at L: $\overset{B}{\doteq}$ is $\doteq^{\mathsf{H}}$, and $\mathsf{A}(\mathsf{true}, \doteq^{\mathsf{H}}) = \{\mathsf{H}\}$, so awareness fails at L. The proof uses obliviousness, and gets it by construction: both slots get a private payload and the wrapper $\cdot + 1^{\mathsf{H}}$, under which every output, `None` included, is unobservable at L. This is needed, because when a slice opens the disk slot emits `Some o` in one run and `None` in the other, and the NOP slot the reverse. The attacker learns nothing from

TABLE I
THE DEVELOPMENT.

file	lines	contents
definitions.v	1281	the calculus, traces, NI, the equivalences
theorems.v	1531	the composition theorems
models.v	1184	the generic model, both policies, Theorem 5, example traces
noninterference.v	1872	the model's equivalences, Theorems 2 to 4
adequacy.v	835	the original definitions, the equivalences with ours, Theorems 2 and 4 in the original's terms

these two slots, and learns that the system is in S only from the others.

The dark and light slots of $\mathcal{O}$ (Definition 3) carry the payload equivalences of the hypotheses of Theorem 4; the white shade of the disk and NOP slots is what this subsection requires.

E. What the proof requires

[TODO: cut this subsection; the intro's second contributions bullet (the requirements) cites it and needs rewording when it goes.]

Read as rules for an implementation, the obligations of this section are sufficient for non-interference: an implementation that follows them can be proved non-interfering in the same way. We do not claim they are necessary; another choice of state equivalence would give other rules.

- The arrival of a secret interrupt may only record that it is pending: by PE, the input step may touch nothing but a private bit (Section V-A).
- When S is entered, and for how long, must be fixed by public events: two related states must agree on whether a secret handler is ready, which takes a filler while the slice is open and the masks while it is closed (Sections V-B and V-C).
- The interrupt controller is split by secrecy: the secret handlers' pending bits are private, and every mask is public, because `initiate_next` branches on the masks (Section V-A).

The sliced policy meets all three; an implementation of it has to preserve them.

VI. MECHANISATION

The development is about 6,000 lines of Rocq 9.0.1, available at https://github.com/Tirore96/NonInterference, using Mathematical Components 2.5.0 [5] with Hierarchy Builder 1.10.1, deriving 0.2.2 and, for the coinductive definitions of the original paper, paco 4.2.3 [6] (Table I). Continuous integration rebuilds it on every change and checks that the paper's results assume nothing beyond the two parameters that fix the security lattice. Identifiers in typewriter font in this paper link to their definitions.

a) Figures: The runs drawn in Figs. 1, 6 and 8 are proved traces of the models, and the state rows of Figs. 6 and 8 are computed by the model's own transition and stated as lemmas (`immediate_states_with_dI`, `sliced_states_with_dI` and their siblings), from which the figures are generated.

b) Types: Input and output types range over an inductive universe `Ty` of codes, interpreted into `Set`, rather than over `Set` directly. This makes reductions invertible, which the negative result needs: refuting non-interference means showing that a trace is *not* admitted.

c) Axioms: The development assumes nothing beyond the two lattice parameters: no classical logic, and no uniqueness of identity proofs. The latter is what dependent elimination on the type-indexed reduction relations would need; we invert reductions through determinism of reduction instead, one lemma per rule (`inv_mapO` and its siblings).

d) Levels: For presentation, this paper reads levels as the two-point lattice. The mechanisation works over any partial order with a least element, and so over any bounded lattice, as the calculus of Rafnsson et al. does, for two reasons: it mechanises the existing theory as it stands, and it lets other models, whose security claims need more levels, build on it. Over such an order, a characterised equivalence must also be monotone downwards: what a level relates stays related below it, and likewise for unobservability. Our models need nothing more. Every equivalence in them treats the bottom level, read as L, apart from all others, so the proofs split on $\ell = \mathsf{L}$ against $\ell \neq \mathsf{L}$, and the second case covers every other level at once; H in $\doteq^{\mathsf{H}}$ and 1^{H} stands for all levels but L.

e) Proof devices: A hook that only the proof needs, such as `restore_invariant`, can hide behaviour. An earlier version of our model relied on it to queue the NOP filler and to return control to the scheduler after a slice; proving Theorem 5 forced both into the interrupt policy, where they now are.

A. Non-interference over finite traces

Our definitions follow Rafnsson et al., with one essential departure. The original defines non-interference as a coinductive simulation over infinite streams of labels, whose clauses apply at the head of a stream. We quantify over finite traces instead, with closure clauses that apply at an arbitrary position (Definition 1). This makes every proof in the development an induction on a trace.

The original definition is written out in `adequacy.v`, and `NI_paper` proves the two equivalent. The proof passes through infinite streams (`NI_SNI`) and rests on two properties: finite and infinite behaviour determine each other, which needs reduction to be deterministic (to assemble finite runs into one infinite run) and never to block (to continue a finite run); and insertion commutes with taking prefixes. Obliviousness, the other definition that quantifies over behaviour, is likewise stated over traces and equivalent to the original (`ObliviousDis_iff`). As a consequence, Theorems 2

and 4 also hold with non-interference read as in the original, hypotheses included (`model_immediate_not_NI_paper`, `model_sliced_NI_paper`).

One further difference is an alternative definition of characterised equivalence. The original adjoins the distinguished value $\bullet$ to every type; we keep the values as they are and make unobservability a predicate, tied to the equivalence by requiring the unobservable values to form one of its classes. `NI_lEquiv` shows that the two give the same notion of non-interference, and this paper uses the original notation throughout.

The composition theorems themselves are proved directly for our definitions.

B. Notation and the mechanised definitions

The paper writes some equivalences differently from their mechanised definitions, and keeps the two forms equal by proof. The mechanisation follows the original for options: `Option` is a type code, classified by `eqmaybe`. Sums, which we add, come there with two equivalences: `eqsum`, under which no sum is unobservable, and `eqsum_L`, under which an `inl` is when its payload is. The paper uses one sum former instead, $\dot{=}^{X} + \dot{=}^{Y}$ (`eqsum_LR`), with $\dot{=}^{\circ}$ (`eqvisible`) and 1^{S} (`unit_equiv`). For each mechanised form, an agreement lemma shows that, at every level, it relates and hides exactly what its paper form does: $\mathsf{eqmaybe}(S, \dot{=})$ is $\dot{=} + 1^{S}$ along the map from options to sums (`eqmaybe_sum_rel`, `eqmaybe_sum_dis`), $\mathsf{eqsum}(\dot{=}^{X}, \dot{=}^{Y})$ is $\dot{=}^{X^{\circ}} + \dot{=}^{Y^{\circ}}$ (`eqsum_sum_rel`), $\mathsf{eqsum_L}(\dot{=}^{X}, \dot{=}^{Y})$ is $\dot{=}^{X} + \dot{=}^{Y^{\circ}}$ (`eqsum_L_sum_rel`), and $(\dot{=}^{\mathsf{L}})^{\circ}$ is $\dot{=}^{\mathsf{L}}$ (`eqvisible_public_rel`). No proof depends on the presentation, and the calculus is the original's, with sums added.

Interrupts are the same technique applied the other way round: there the mechanisation was changed to match the presentation. The input type is the sum $Unit + Unit$, with the disk interrupt on the `inl` side, and `in_equiv` is $\dot{=}^{\mathsf{H}} + \dot{=}^{\mathsf{L}}$. The earlier definition, over a two-constructor type, is kept as `in_equiv_interrupt`, and `in_equiv_interrupt_rel` and `in_equiv_interrupt_dis` show that the two agree.

VII. Related Work

a) Non-interference: Non-interference goes back to Goguen and Meseguer [2]; Sabelfeld and Myers survey language-based approaches [1]. Timing channels in programs have been closed by padding transformations that equalise the running time of secret-dependent branches [3]; the sliced policy applies the same idea to handler runs, with the NOP handler as padding.

b) Concurrency and scheduling: How the scheduler affects the security of multi-threaded programs has been studied since Smith and Volpano [7]. Sabelfeld and Sands give a probabilistic, timing-sensitive non-interference for multi-threaded programs [8], Mantel and Sudbrock a notion of security that holds for a whole class of schedulers [9], and Stefan et al. address termination and timing channels in a concurrent information-flow system [10]. These works treat the scheduler as given and ask which programs are secure under it. We treat the interrupt mechanism, which decides when handlers preempt the scheduled process, as the object to be made secure, for any non-interfering scheduler and user processes.

c) Timing-channel mitigation: Predictive mitigation delays outputs to a schedule predicted from public information, bounding what their timing leaks [11], also in a language-based setting [12]. Our requirement has the same shape, applied to the interrupt mechanism: when secret handlers run, and for how long, must be fixed by public events. The sliced policy meets it exactly, so nothing leaks, at the cost of reserving handler time whether or not it is used.

d) Composable timing-sensitive non-interference: Our proofs rest on the calculus of Rafnsson et al. [4], whose definitions are timing-sensitive by construction and whose theorems compose. We mechanise the calculus and its composition theorems, and apply them to a model of interrupt handling.

e) Verified operating systems: Murray et al. formulate non-interference for operating system kernels [13] and prove it for seL4 [14], whose proof leaves timing channels out of scope; Cock et al. measure the timing channels that remain in seL4 and their mitigation [15]. Time protection [16] argues that an operating system must partition time as well as space, and includes the partitioning of interrupts among its mechanisms[TODO: check this characterisation against [16]]; our sliced policy is an instance of that idea in a small model. What we add is a timing-sensitive, compositional, mechanised proof that the mechanism is secure, for any non-interfering scheduler and user processes.

VIII. Discussion and Conclusion

a) The model and a real system: [TODO: write: how the model compares with a real system (time, interrupt hardware, devices and cores) and the cost of the sliced policy.]

b) Future work: [TODO: write: richer models, implementing the policy, and how to ensure that a real implementation is safe.]

c) Conclusion: A secret interrupt leaks through the schedule alone: the attacker learns when its handler runs and for how long. To close the channel, both must be fixed by public events, and only which secret handler runs may depend on the secret. Servicing secret handlers only in time slices of fixed length, opened by the public timer handler and padded by a dummy handler, achieves this. We proved the leak and its absence in Rocq, the latter for any non-interfering scheduler and user processes, by composing generic non-interference theorems over the structure of the model. Both results hold for an attacker who sees the whole schedule and for the user view, which shows only the two user processes. The proof did more than check the policy: it fixed the classification of the interrupt controller bit by bit, and it separated hiding what a handler computed from hiding that it ran.

Acknowledgment

[TODO: draft for the IEEE/CSF policy on AI-generated content; confirm the scope and the sections, and name the model version.] Following the IEEE policy on AI-generated content, we disclose that Claude (Anthropic), an AI assistant, was used to draft and revise text and figures throughout the paper, and to develop parts of the Rocq mechanisation. The authors reviewed all AI-generated content and take full responsibility for it.

References

[1] A. Sabelfeld and A. C. Myers, "Language-based information-flow security," *IEEE Journal on Selected Areas in Communications*, vol. 21, no. 1, pp. 5–19, 2003.

[2] J. A. Goguen and J. Meseguer, "Security policies and security models," in *IEEE Symposium on Security and Privacy*, 1982, pp. 11–20.

[3] J. Agat, "Transforming out timing leaks," in *Principles of Programming Languages (POPL)*. ACM, 2000, pp. 40–53.

[4] W. Rafnsson, L. Jia, and L. Bauer, "Timing-sensitive noninterference through composition," in *Principles of Security and Trust (POST)*, ser. Lecture Notes in Computer Science, vol. 10204. Springer, 2017, pp. 3–25.

[5] A. Mahboubi and E. Tassi, *Mathematical Components*. Zenodo, 2022.

[6] C.-K. Hur, G. Neis, D. Dreyer, and V. Vafeiadis, "The power of parameterization in coinductive proof," in *Principles of Programming Languages (POPL)*. ACM, 2013, pp. 193–206.

[7] G. Smith and D. Volpano, "Secure information flow in a multi-threaded imperative language," in *Principles of Programming Languages (POPL)*. ACM, 1998, pp. 355–364.

[8] A. Sabelfeld and D. Sands, "Probabilistic noninterference for multi-threaded programs," in *Computer Security Foundations Workshop (CSFW)*. IEEE, 2000, pp. 200–214.

[9] H. Mantel and H. Sudbrock, "Flexible scheduler-independent security," in *European Symposium on Research in Computer Security (ESORICS)*, ser. Lecture Notes in Computer Science, vol. 6345. Springer, 2010, pp. 116–133.

[10] D. Stefan, A. Russo, P. Buiras, A. Levy, J. C. Mitchell, and D. Mazières, "Addressing covert termination and timing channels in concurrent information flow systems," in *International Conference on Functional Programming (ICFP)*. ACM, 2012, pp. 201–214.

[11] A. Askarov, D. Zhang, and A. C. Myers, "Predictive black-box mitigation of timing channels," in *Computer and Communications Security (CCS)*. ACM, 2010, pp. 297–307.

[12] D. Zhang, A. Askarov, and A. C. Myers, "Language-based control and mitigation of timing channels," in *Programming Language Design and Implementation (PLDI)*. ACM, 2012, pp. 99–110.

[13] T. Murray, D. Matichuk, M. Brassil, P. Gammie, and G. Klein, "Noninterference for operating system kernels," in *Certified Programs and Proofs (CPP)*, ser. Lecture Notes in Computer Science, vol. 7679. Springer, 2012, pp. 126–142.

[14] T. Murray, D. Matichuk, M. Brassil, P. Gammie, T. Bourke, S. Seefried, C. Lewis, X. Gao, and G. Klein, "seL4: From general purpose to a proof of information flow enforcement," in *IEEE Symposium on Security and Privacy*, 2013, pp. 415–429.

[15] D. Cock, Q. Ge, T. Murray, and G. Heiser, "The last mile: An empirical study of timing channels on seL4," in *Computer and Communications Security (CCS)*. ACM, 2014, pp. 570–581.

[16] Q. Ge, Y. Yarom, T. Chothia, and G. Heiser, "Time protection: The missing OS abstraction," in *European Conference on Computer Systems (EuroSys)*. ACM, 2019, pp. 1:1–1:17.